

OPUS.NET Design Conversation

Complete conversation transcript - 26 August 2026

Scope: server-side persistence, in-memory domain model, flat facade function tier, and TCP/IP server architecture.

User - Turn 1

MEL and ENTER needs a .NET multi-tier framework. let us start the process of designing this. i call it OPUS.NET. it will have a object-network database persistence layer, a middle-tier layer where business rules close to the physical server machine will be handled including queries and multi-machine distributed storage and distributed transaction management will be handled, a server-side facade into the middle-tier domain model, where this facade packs all requests in a binary TCP/IP protocol for the client-side of the domain model to use and finally a way to choose how much of the server side domain model content should be downloaded into the client side domain model. engage

Assistant - Turn 2

Yes. The architecture is coherent, but the key is to define OPUS.NET as a distributed domain-object runtime, not merely a networking or ORM framework. I would start by separating the responsibilities sharply enough that MEL and ENTER can share the same infrastructure without either product leaking into the framework.

Yes. I think OPUS.NET should be designed as a reusable distributed object-network runtime for .NET, with MEL and ENTER simply becoming domain applications built on top of it.

The most important architectural decision is that OPUS.NET should not expose database concepts to the application. Applications should operate on live domain objects and relationships. Persistence, distribution, synchronization, serialization, caching, transactions, and client/server transport happen underneath that domain model.

INITIAL ARCHITECTURE

APPLICATION DOMAIN

MEL / ENTER / OPUS ERP / etc.

6. CLIENT DOMAIN RUNTIME

Local object network / cache / lazy loading / change set

5. OPUS.NET BINARY PROTOCOL + SERVER FACADE

TCP/IP / authentication / requests / responses / streams

4. SERVER DOMAIN RUNTIME

Domain objects / business rules / object lifecycle

3. DISTRIBUTED MIDDLE TIER

Queries / transactions / routing / shards / coordination

2. OBJECT NETWORK ENGINE

Object identity / relations / serialization / indexing

1. PHYSICAL STORAGE

GUID -> binary object blob / indexes / transaction journal

This is slightly different from the conventional UI -> API -> service -> ORM -> SQL architecture. That is intentional.

OPUS.NET's fundamental abstraction becomes: A persistent distributed network of addressable .NET objects.

1. Object-network persistence

Your original concept of an object fundamentally being:

GUID

SerializedBlob

is very attractive as the canonical storage representation.

For example:

```
8FAD2B6A-... -> [serialized Customer object]
91BDE388-... -> [serialized Order object]
CB11994E-... -> [serialized Product object]
```

Relations are simply GUID references. OPUS.NET reconstructs an ordinary .NET object network in memory.

A pure GUID + Blob database is excellent for loading objects by identity, but poor for queries. We should keep canonical object storage simple while maintaining derived TypeIndex, PropertyIndex, RelationIndex, FullTextIndex, TransactionJournal, and ShardDirectory structures. The object blob is truth; indexes are accelerators.

2. Object Network Engine

The runtime can maintain object identity and state such as Proxy, Loaded, Modified, Deleted, Detached, and Faulted. Lazy retrieval can provide object-level virtual memory over the network.

3. Distributed middle tier

This handles object location, query execution, indexes, transactions, locks/MVCC, shard routing, replication, machine membership, caching, failure recovery, authorization, and server-authoritative business rules.

The client never needs to know which server owns an object. An internal ObjectDirectory could conceptually map GUID -> StoragePartition.

4. Transactions

Separate local transactions from distributed transactions. An OpusTransaction API could eventually coordinate changes across machines, while object placement should try to colocate objects commonly modified together.

5. Server-side domain model

MEL might contain Campaign, Customer, EmailList, EmailAddress, ProofReader, Delivery, Payment. ENTER might contain Creator, Consumer, Product, DomainObject, Collection, Subscription, Purchase, License. Server-side methods are trusted; client-side objects are not trusted.

6. Server Facade

The facade is the security boundary between the Internet and the server object network. It accepts protocol messages and turns them into middle-tier operations such as GET_OBJECT, GET_OBJECTS, QUERY, CALL_METHOD, BEGIN_TRANSACTION, COMMIT_TRANSACTION, ROLLBACK_TRANSACTION, SUBSCRIBE, UPLOAD_STREAM, DOWNLOAD_STREAM.

Selective client domain models can be treated as Domain Projection: the server has the full domain graph while each client receives a projected graph. Loading modes could include SERVER ONLY, REFERENCE, LAZY, and EAGER.

Queries should execute server-side. The client-side domain runtime can contain ObjectCache, IdentityMap, ProxyManager, ChangeTracker, QueryClient, ProtocolClient, StreamManager, and TransactionContext.

Changes from client to server can be represented as change sets. Streams should remain separate from normal domain blobs.

The conceptual model can be reduced to OBJECT, RELATION, PROJECTION, QUERY, and TRANSACTION.

A possible solution structure:

- OPUS.Core
- OPUS.Serialization
- OPUS.ObjectModel
- OPUS.Storage
- OPUS.Storage.Indexing
- OPUS.Storage.Transactions
- OPUS.Distributed
- OPUS.Distributed.Clustering
- OPUS.Distributed.Transactions
- OPUS.Query
- OPUS.Protocol
- OPUS.Server
- OPUS.Client
- OPUS.Security
- OPUS.Tools
- OPUS.Tests

Applications then become ENTER.Domain / Server / Client and MEL.Domain / Server / Client.

The next thing I would design is the OPUS Object Model contract first: object identity, references, versions, serialization, states, lazy proxies, and projections.

User - Turn 3

i understand the basic architecture. let us begin with the server side persistence framework, the DB itself. i see the simplest solution being a object network db where each object (identified with a guid and the object content serialized as a blob) is a file on the file system where the filename is the guid and the postfix ".bin" all stored in a single folder. the middle tier that knows the guid of the object (stored in memory in the server-side domain model) simple accesses each file and CRUD this file. analyze

Assistant - Turn 4

Yes. As a first persistence engine, this is extremely attractive because it reduces the database to the smallest possible abstraction:

Object identity = GUID
Persistent representation = serialized binary blob
Physical address = <root>\<GUID>.bin

The core contract could be as small as:

```
interface IObjectStore
{
    bool Exists(Guid id);
    byte[] Read(Guid id);
    void Create(Guid id, byte[] data);
    void Update(Guid id, byte[] data);
    void Delete(Guid id);
}
```

Above that, serialization turns it into a typed object store:

```
T Load<T>(Guid id);
void Save<T>(Guid id, T obj);
```

Its biggest strength is that object identity and physical persistence match almost perfectly. There is no ORM, table decomposition, joins, or foreign keys. The persistence layer does not need to understand the meaning of relationships.

CRUD is simple: serialize/write for Create; read/deserialize for Read; serialize/replace for Update; delete file for Delete.

Several issues should be considered before calling it finished:

1. A single folder eventually becomes a scalability problem. Preserve GUID -> file logically, but allow physical partitioning such as Objects\8f0c\<GUID>.bin.
2. Never update the .bin file in place. Write GUID.tmp, flush, then atomically rename/replace GUID.bin to avoid half-written objects after crashes.
3. Consider metadata around the blob: magic, format version, object ID, type ID, object version, payload length, checksum.
4. Do not blindly couple persistence to CLR serialization. OPUS.NET may eventually need explicit serialization semantics.
5. GUID references should remain GUIDs on disk. One persistent object = one file. Do not recursively serialize the entire object graph.
6. Reads are conceptually efficient: GUID -> path -> OS file cache -> deserialize.
7. Querying is the major limitation if the GUID is not already known. Separate object storage from query indexing.
8. Concurrency may need object-level coordination in a concurrent server, though your later architecture may simplify this.
9. File deletion may eventually need tombstones or recoverable deletion for transactions.
10. Separate Object Serializer, Object Store, and Physical File Store so alternative stores can be introduced later.

The fundamental idea is sound. I would modify 'every object is a .bin file in one folder' to 'every persistent object has exactly one independently addressable object file, identified by GUID; the file store is free to partition those files physically.'

User - Turn 5

source code changes that requires and update of the schema on file will be a manual process to transform v1 domain model schema to v2 domain model schema, but for me not a big problem since i going to develop the domain model and the ui with no DB persistence first, and when these components are stable, engage the DB storage. the server side domain model only accepts one request at a time, blocking (wait block the singleton thread) and concurrency is handled at the client. i have the preliminary design for the lowest server side layer on top of the persistence layer.

```
class ServerDomainModelStorageWithNETSerialization
    T Load<T>(Guid id);
        if IObjectStore.Exists(id) then
            object T = Deserialize(IObjectStore.Read(id))
        else
            throw ObjectDoNotExistException
        endif
    void Save<T>(Guid id, T obj);
        if IObjectStore.Exists(id) then
            IObjectStore.Update(id, Serialize(T, obj))
        else
            IObjectStore.Create(id, Serialize(T, obj))
        endif
    void Delete(Guid id);
        if IObjectStore.Exists(id) then
            IObjectStore.Delete(id)
        else
            throw ObjectDoNotExistException
        endif

class ServerDomainModelStorageWithBinarySerialization
    T Load<T>(Guid id);
        if IObjectStore.Exists(id) then
            object T = DeserializeWithSchema(schemaObject, IObjectStore.Read(id))
        else
            throw ObjectDoNotExistException
        endif
    void Save<T>(Guid id, T obj);
        if IObjectStore.Exists(id) then
            IObjectStore.Update(schemaObject, id, SerializeWithSchema(schemaObject, T, obj))
        else
            IObjectStore.Create(schemaObject, id, SerializeWithSchema(schemaObject, T, obj))
        endif
    void Delete(Guid id);
        if IObjectStore.Exists(id) then
            IObjectStore.Delete(id)
        else
            throw ObjectDoNotExistException
        endif
```

Assistant - Turn 6

This is a good simplification. Given your execution model - one server-domain-model request at a time - I would deliberately keep this layer very small and avoid adding locking, MVCC, transaction coordination, or schema migration machinery here.

The two classes are conceptually correct, but storage semantics and serialization strategy can be separated to avoid duplicated logic.

IObjectStore knows GUIDs and bytes.

IObjectSerializer knows objects and bytes.

ServerDomainModelStorage knows how to combine them.
Domain Model knows objects.

A common ServerDomainModelStorage can implement Load, Save, and Delete, while NetObjectSerializer and OpusBinarySerializer implement IObjectSerializer.

The filesystem should know nothing about schemas. IObjectStore.Update/Create should receive id and bytes only; schema belongs entirely to the serializer.

Your manual schema strategy makes sense: when an intentional schema change occurs, a separate migration utility can load V1 with the V1 schema, transform V1 objects to V2, and serialize using the V2 schema.

Your single-thread server model also eliminates the usual Exists/Create race inside the server domain model, provided only the OPUS.NET server process modifies the object-store directory.

Load<T>(Guid id) is acceptable because the caller already knows the type. Generic polymorphic loading can be deferred unless actually needed.

The lowest stack becomes:

```
SERVER DOMAIN MODEL
|
ServerDomainModelStorage
|
+-- IObjectSerializer
|   +-- NET Serializer
|   +-- OPUS Binary Serializer -> DomainSchema
|
+-- IObjectStore
    +-- FileObjectStore -> GUID.bin files
```

Responsibilities:

Domain Model = behavior and relationships

ServerDomainModelStorage = persistence semantics

Serializer = object <-> bytes

ObjectStore = GUID <-> bytes

FileObjectStore = GUID <-> physical .bin file

User - Turn 7

So, i could implement SQLServerObjectStore implementing IObjectStore for implicit transactions and crash recovery?

Assistant - Turn 8

Exactly. That is one of the main benefits of keeping IObjectStore narrow.

You can have FileSystemObjectStore and SQLServerObjectStore implementing the same interface.

For SQL Server, the physical schema could be nearly minimal:

```
CREATE TABLE Objects
(
    ObjectID UNIQUEIDENTIFIER NOT NULL PRIMARY KEY,
    Data VARBINARY(MAX) NOT NULL
);

Exists(Guid) -> SELECT existence
Read(Guid) -> SELECT Data
```

```
Create(Guid, bytes) -> INSERT  
Update(Guid, bytes) -> UPDATE  
Delete(Guid) -> DELETE
```

SQL Server gives the lowest storage layer mature transactional durability and crash recovery without changing the OPUS.NET object model.

One important distinction: SQL Server gives an implicit transaction around each individual statement. A domain operation that performs Save(customer), Save(order), Delete(invoice) is still three operations unless wrapped in an explicit SQL transaction. Therefore individual CRUD operations can be atomic first, while explicit multi-object transactions can be added later.

OPUS.NET is not becoming relational. SQL Server is merely a reliable GUID -> VARBINARY(MAX) storage engine. OPUS.NET retains ownership of the object model, relationships, serialization, and querying semantics.

User - Turn 9

let us analyse further under these conditions: the server side domain model is completely loaded in memory (except from the additional BLOB storage mechanism that will be available on the side of the domain model) and based on f.ex domain model types distributed across multiple server machines, hard-coded or configured in a server-side configuration file, etc something, like even a frontend server-side load balancing mapper to a secondary layer of multiple domain model server machines....bla.bla. this and the following: how do we handle queries? first thing that pops up is that the facade of the server-side domain model offers explicit Find-methods as a flat list on a singleton object, but also more advanced such as server-side domain model uses an in-memory index to speed things up. analyse

Assistant - Turn 10

Under those conditions, queries become much simpler than in a conventional database architecture because the server-side domain model itself is the primary queryable state. Persistence is mainly for restart/recovery, not normal read access.

Persistent object store = restart/recovery
In-memory domain model = live authoritative state
Query subsystem = navigation/indexing over live state

Normal queries should not hit persistence.

The simplest first version is explicit query methods exposed by the server-side facade: FindCustomer, FindCustomerByEmail, FindCustomersByCountry, FindOrdersByCustomer, FindProducts, etc. The facade becomes the query contract between client and server.

This gives a strong abstraction boundary: FindCustomerByEmail can initially scan a collection, later use a Dictionary index, later route to another server, without changing the client contract.

Domain queries should be separated from query mechanics. OPUS.NET can provide generic in-memory index mechanics while the application exposes domain-specific Find methods.

A typed GUID collection can act as a natural primary index. Secondary indexes can be added where necessary, such as Email -> Customer or Country -> list of Customers. Indexes can be rebuilt at startup because the authoritative state is the domain objects themselves.

Distribution changes query routing, not query semantics. A frontend server can route Customer queries to one domain server, Product queries to another, Order queries to another. Cross-partition queries can later be

coordinated, but are not required in v1.

Three query levels are useful conceptually:

1. Domain-specific Find methods using scans.
2. Same Find methods accelerated by indexes.
3. Same Find methods implemented through distributed routing/fan-out.

A distinction between precise Find methods and broader Search methods with typed criteria objects can avoid the complexity of remote generic LINQ.

Because the domain model is fully loaded, persisted GUID relationships can be resolved to direct object references after startup. Large BLOBs remain outside the loaded graph and are demand-loaded/streamed.

The key principle: Queries operate on the live in-memory domain model. Persistence is not queried. Domain-specific facade methods define query semantics; indexes are transparent accelerators; distribution is transparent routing.

User - Turn 11

i agree that the registry is necessary for reconstructing the object network after startup, but would not the registry provide a typed object? how do i know what type the object has if retrieved from the registry? my approach as a hierachial domain model with a known root (such as a singleton "Application" object and underneath that the basic practical typed objects, such as a Customer hashlist. am i wrong here? would not storing objects directly in the index/registry/hashtable/collection be smaller and faster then the indirect mapping? you seem excited and are getting slightly ahead of the design process here.

Assistant - Turn 12

You are not wrong. Given the architecture you are describing, your hierarchical root-object approach is probably simpler than a generic global Guid -> object registry.

If the server domain model is fully loaded and structurally known, you can have:

```
Application
{
    Dictionary<Guid, Customer> Customers;
    Dictionary<Guid, Order> Orders;
    Dictionary<Guid, Product> Products;
}
```

Then the collection itself is both registry and typed index. Customer c = Application.Customers[id] already answers both what object has this GUID and what type it is.

A generic global registry is only needed if OPUS.NET must resolve a GUID without any type information. Your current domain model does not necessarily require that. If an Order refers to a Customer, the relationship is logically typed, and reconstruction can resolve order.Customer from application.Customers[customerId].

Startup can load the Application singleton, load typed collections, then resolve relationships.

Storing actual object references directly in in-memory indexes is also preferable when the indexes are not persisted. For example Dictionary<string, Customer> CustomersByEmail avoids email -> GUID -> Customer indirection.

The rule I would adopt is: Persist relationships as GUIDs; once loaded, resolve them to normal typed object references. In-memory indexes may point directly to typed objects.

I would retract the claim that a universal `ObjectRegistry<Guid, object>` is necessary at this stage. Your root-domain hierarchy already provides the registry function in a more strongly typed and domain-natural way.

User - Turn 13

i can not see value in a non-typed registry since in the end, the client ui must map data to a user-exposed structure, even free-form ui interface needs this. f.ex the binary content of an untyped object, will this be understood by the end user. the only use i see is some kind of diagnostics where the untyped is examined though reflection, but what will that do to practical enterprise business use?

Assistant - Turn 14

I agree with your conclusion. For this architecture, a global untyped registry has little practical value and mostly introduces indirection without solving a real business problem.

Somewhere in the chain, data must acquire domain meaning. A user interacts with Customer, Invoice, Campaign, Product, Payment, etc., not an arbitrary blob. Even a dynamic UI needs metadata describing fields, display, operations, and relationships.

`Dictionary<Guid, object>` does not remove typing; it postpones it. Eventually code still needs a cast, reflection, or equivalent metadata.

A stronger OPUS.NET principle is appropriate: Every persistent business object belongs to a known domain type, and every in-memory object is reached through a typed part of the domain model.

Reflection can still be useful for diagnostics, serialization, UI generation, schema inspection, logging, or tooling without requiring the domain model to be stored untyped.

Dynamic UI is not the same as an untyped domain. A dynamic UI can be generated from a strongly typed domain model.

The global `ObjectRegistry<Guid, object>` can therefore be dropped at this stage. Typed collections under Application can serve ownership/containment, GUID lookup, iteration, startup reconstruction, and domain navigation. Additional query indexes can point directly to typed objects.

This keeps OPUS.NET object-oriented from persisted identity to business logic to UI.

User - Turn 15

my idea is that the server-side domain model is NOT exposed directly to the client, but every FUNCTION needed by the client (`FindByEmail`, `SaveCustomer`, etc) is offered as a facade, i.e a flat list of functions. the implementation of these function will manage transactions, load balancing, reader/writer lock, etc

Assistant - Turn 16

Yes. That is a clearer boundary.

The client should not navigate or manipulate the server-side domain model directly. The server-side domain model is an implementation detail. The client sees only a facade contract consisting of explicit functions.

The facade defines what operations are legal; the domain model defines how those operations work.

A facade function such as SaveCustomer can internally route to the correct domain server, acquire write access, begin a transaction, validate/update the domain object, update indexes, persist changes, commit, release access, and return success. FindCustomerByEmail can route, acquire read access, query an in-memory index, map the result, release access, and return.

The client knows none of that.

The flat list is the service contract. It does not require one giant facade class. Internal source organization can be modular while the remote surface remains flat.

The architectural statement is: The server-side domain model is private. Clients never access it directly. All remote interaction occurs through explicit facade functions. Each facade function can own routing, concurrency control, transaction scope, authorization, domain invocation, persistence coordination, and result projection as required.

User - Turn 17

what is DTD? the reason for defining a flat list of functions is that this has to be mapped to a binary TVP/IP protocol interfacing the client and the server and flat is the simplest rather than an unneeded hierarchy of functions that would be the result of namespaces. the namespace organization will be implicit in the function naming, f.ex "FindOrderOwnedCustomer". i did not specify that the functions would be owned by one facade class, but a single tier available to the tcp/ip server host entry point (as seen by the client). you are still a bit ahead of the design process.

Assistant - Turn 18

DTD usually means Document Type Definition, historically used with XML to define what elements and structure an XML document may contain. It is not relevant to the OPUS.NET design unless XML enters the picture later.

Your clarification changes how the facade should be framed. You are not proposing one giant Facade class or a hierarchical service namespace. You are defining a single callable server tier, exposed through the TCP/IP host entry point, where every callable operation has a unique flat function identity.

From the client point of view the contract is simply names such as FindCustomerByEmail, FindOrderOwnedCustomer, SaveCustomer, DeleteOrder, CreateCampaign, etc. Each name can implicitly carry domain grouping.

The binary protocol can map a FunctionId and RequestPayload to a flat dispatch table FunctionId -> implementation. The implementation may physically live across many classes, assemblies, machines, or domain partitions. That organization is invisible to the client and protocol.

Flat protocol surface does not imply flat source-code organization.

The server exposes one flat set of uniquely named callable functions. The TCP/IP protocol maps requests directly to those functions. How those functions are implemented or grouped internally is not part of the client contract.

User - Turn 19

we now have the persistence layer, the server-side domain model layer and the function list facade layer, this might seem like a lot of layers, but imagine the testability of this design compared to merging functionality in fewer layers, which i believe is more common in competing frameworks today. comment on that and let us continue to the tcp/ip layer server-side

Assistant - Turn 20

I agree with the layering. It is more layers than many mainstream frameworks use, but each layer has a narrow responsibility and therefore a clean test boundary.

Persistence can be tested with no business model. The domain model can be tested in memory. Each facade function can be tested without opening a socket. The TCP/IP layer can then be tested separately as binary request -> decode -> dispatch -> encode response.

Layers become harmful when they merely forward calls without adding a meaningful contract. Here the contracts are distinct enough to justify themselves.

The next server-side stack is:

```
TCP/IP Server Host
-> Function-list facade
-> Server-side domain model
-> Persistence
```

The TCP/IP server initially needs only to accept a connection, read a complete request, decode function identifier and parameters, dispatch a facade function, encode the result, and write the response.

A minimal request can contain FunctionId, PayloadLength, Payload; response can contain Status, PayloadLength, Payload. The TCP host need not understand Customers or Orders.

One issue to decide is whether one TCP connection handles one outstanding request at a time or multiple outstanding requests. Given the current model, a simple synchronous request -> worker -> facade -> response flow is enough to establish the architecture.

User - Turn 21

again, versioning belongs to the developer using a practical software configuration management methodology with supporting tools to convert from an older version to the recent version. now for the tcp/ip server interfacing directly on the network. there will be one listener thread. the listener reads the first byte of the incoming request package and if this byte maps the boolean value of true of acquires a write-loc, if it is false, it acquires a read-lock. it then hands the incoming package to a worker thread on from a thread pool. the worker thread then reads the next section of the incoming package and identifies which facade function should be called, with the now deserialized input data as function parameters, it calls the facade function, wait for the function call to return and serializes the result into a response and sends it back to the client (knowing the client-side end-point) and goes back to the thread pool. analyse

Assistant - Turn 22

This is a coherent next layer, with two important corrections while preserving the basic design.

The TCP layer establishes the concurrency boundary before a facade function touches the domain model.

I would move actual lock acquisition from the listener to the worker. If the listener blocks waiting for a write lock, it cannot continue servicing incoming connections. The listener can classify/dispatch while the worker owns the lock.

A ReaderWriterLock changes the earlier single-request rule slightly: multiple read-only facade operations may execute concurrently, while writes execute exclusively.

The first byte should not be trusted as the sole authority for whether a function is read or write. The server's function metadata should know the access mode and can verify the client flag or derive it from FunctionId.

A worker lifecycle can be:

1. Receive complete framed request.
2. Identify facade function.
3. Determine/verify READ or WRITE.
4. Deserialize parameters.
5. Acquire appropriate domain lock.
6. Invoke facade function.
7. Wait for return.
8. Release domain lock.
9. Serialize returned value.
10. Send response.
11. Return worker to pool.

Release the domain lock before serialization/transmission if the returned result is already detached/safe.

TCP needs request framing because it is a byte stream. A package length can precede the access mode, function ID, and serialized parameters.

The response normally goes back over the same accepted TCP connection; no second connection to the client's endpoint is required.

The resulting separation is:

TCP layer = network framing + dispatch + concurrency gate

Facade tier = callable server operations

Domain model = enterprise/business behavior

Persistence = durable state

User - Turn 23

is it possible for the listener to simply hand the unparsed incoming package to the worker thread, where the worker thread reads the first byte and acquires a readerwriterlock on a generic singleton object or must the listener to this, in any case, analyse and describe a full setup

Assistant - Turn 24

Yes. The listener can be kept extremely small and hand the complete unparsed request package to a worker. In this design that is cleaner than having the listener interpret the read/write byte.

The distinction is between network framing and application parsing. The listener must know enough to determine where one TCP message ends, because TCP is a byte stream, but it need not understand the contents beyond that.

A request package can contain:

Int32 PackageLength

Byte AccessMode

Int32 FunctionId

Byte[] SerializedParameters

The listener reads PackageLength and exactly that many bytes, then hands the byte array to a worker without interpreting AccessMode, FunctionId, or parameters.

The worker reads `AccessMode` and `FunctionId`, deserializes parameters, acquires a `ReaderWriterLockSlim`, invokes the facade synchronously, releases the lock, serializes the result, sends it, and returns to the pool.

The lock can be a single generic singleton synchronization object shared by the entire server-side domain-model instance. A static holder is conceptually simple, although an instance-owned lock would allow multiple independent domain models in one process.

READ facade requests acquire a shared/read lock. WRITE facade requests acquire an exclusive/write lock.

The server should still validate the requested access mode against metadata for the facade function, or eventually derive access mode from the function ID.

A useful full worker sequence:

1. Receive complete opaque request package from listener.
2. Read `AccessMode` byte.
3. Read `FunctionId`.
4. Look up facade function metadata.
5. Verify `AccessMode`.
6. Deserialize parameters.
7. Acquire reader or writer lock.
8. Invoke facade function synchronously.
9. Wait for return.
10. Release lock.
11. Serialize return value.
12. Send response on the same TCP connection.
13. Return worker to thread pool.

If connections are persistent, the actual listener should ideally only `Accept()`. A connection handler owns each accepted socket, performs framing, keeps the connection alive, and hands complete packages to workers.

So:

LISTENER = establishes TCP connections

CONNECTION LAYER = owns connected socket, performs framing, keeps connection alive, hands packages to workers

WORKER = interprets protocol, acquires static reader/writer lock, calls facade, creates response, sends through originating connection

FACADE = knows nothing about sockets or locking

DOMAIN MODEL = knows nothing about TCP

PERSISTENCE = knows nothing about domain storage

User - Turn 25

how does the worker thread know where to send the response to (is the client end-point an implicit element of the tcp/ip protocol)? i do prefer the static lock holder as i see that this way of doing it does not cross into the implementation of the facade layer. is this true? explain further with a use case on the benefit of persistent connection layer rather than the listener reading the byte array

Assistant - Turn 26

Yes. With TCP, the worker does not need the client endpoint encoded inside the application package if the response is sent back over the same accepted TCP connection.

When the server listener accepts a client connection, the operating system gives the server a connected socket representing both ends of the TCP session. The server-side socket already knows the remote IP address and

port as part of TCP connection state.

If the listener or connection handler passes the connected socket together with the request bytes to the worker, the worker can send the response through that socket. The worker should therefore receive Connection + Request rather than only Request.

Regarding the static lock holder: yes, it can keep the locking concern entirely above the facade implementation. The worker acquires ServerDomainModelLock, calls the facade, then releases the lock. The facade need not know the reader/writer lock exists. This also improves testability because facade functions can be called directly in tests without TCP or synchronization infrastructure.

The assumption should be explicit: one static lock protects one logical server-side domain model within that process.

Persistent connections are useful because an interactive client can connect once and perform many facade calls rather than connect/request/disconnect for every operation. This avoids repeated TCP establishment overhead and, if TLS is later used, repeated security handshakes. It also creates a natural client session channel.

A persistent connection does not mean the listener must parse requests. The cleaner split is:

Listener: Accept() new connections only.

Connection handler: read package length, read complete package, submit connection + byte[] to worker, repeat.

Worker: parse request, determine read/write, deserialize arguments, acquire static lock, invoke facade, release lock, serialize result, send through connection.

Use case: three ENTER clients A, B, and C. If the only listener thread accepts A and then waits while reading a large or slow request from A, B and C may wait to be accepted. With a connection layer, the listener accepts A and hands its socket to a connection handler, immediately returns to Accept(), then accepts B and C. Each connection can receive bytes independently and submit complete requests to the shared worker pool.

The server-side TCP architecture therefore becomes:

LISTENER - establishes TCP connections

CONNECTION LAYER - owns connected socket, performs framing, keeps connection alive, hands complete packages to workers

WORKER - interprets protocol, acquires static reader/writer lock, calls facade, creates response, sends through originating connection

FACADE - knows nothing about sockets or locking

DOMAIN MODEL - knows nothing about TCP

PERSISTENCE - knows nothing above domain storage

A low-level connection abstraction should also hide TCP details such as partial Send() and Receive() operations by providing SendAll(byte[]) and ReadExactly(n).

User - Turn 27

generate a pdf of this entire chat to download